-Universidad Arturo Jauretche

Ingeniería en informática

INFORME TÉCNICO

Arcoíris

Metodologías de la programación II

INTEGRANTES DEL EQUIPO

- **Estudiante 1:** Pastrana Matias Ariel
- **Estudiante 2:** Guillerme Nicasio Aguirre
- **Estudiante 3:** Abraham Cesar Torres

INFORMACIÓN ACADÉMICA

Docente: Christian Nahuel BOTTA

Comisión / Grupo: 1

Fecha de Entrega: [DD/MM/AAAA]

Problemática

La pinturería Arcoíris necesita tener un stock por marca, colores, precios y tamaños. Cuando un cliente quiere comprar, se busca en una carpeta las boletas de compra y se calcula el

precio sobre el precio de costo. Esto como herramienta para el uso del sector de atención al público.

Registra marcas, colores y precios. De esta forma se conoce que stock de cada articulo y si hay se hace el pedido para reposición. Se actualizan los precios en caso de incrementos y/o promociones y se tiene un stock actualizado.

Se busca en un principio usarlo como aplicación de consola. Usando la metodología xp y en lenguaje Python versión 3.1

Implementación de las clases utilizadas.

Clase	Responsabilidad	Atributos / Métodos Clave
Usuario	Gestiona la información y la identificación de los usuarios en el sistema.	- nombre: string - dni: string - id: string + mostrar()
GestorProductos	Controla la lógica de negocio principal, búsqueda, cálculo de precios y emisión de tickets.	- ListaProductos: List<Producto> + agregarProducto(p: Producto) + aplicarDescuento(p, d): float + verificarStock(p): bool + buscarProducto(id): Producto
Producto	Representa la entidad de dominio con las propiedades del artículo y su stock individual.	- id: int, nombre, marca: string - precio: float, stock: int + actualizar_stock(cant: int) + calcular_precio(): float
Ticket	Registra el detalle de ítems comprados y gestiona el cálculo e impresión del comprobante.	- items: List<(Producto, int)> + agregarP(pro, can) + calTotal(): float + impriTiket()
Descuento	Representa las reglas de bonificación o promociones aplicables a los precios.	- tipo: string - porcentaje: int + aplicar(precio: float): float
Stock	Monitorea la disponibilidad general de inventario y la emisión de alertas de reabastecimiento.	- cantidadDisponible: int - alerta: bool + verifica(): bool

Clase	Responsabilidad	Atributos / Métodos Clave
Pedido	Administra la orden de compra realizada, su estado y la lista de productos solicitados.	- IdPedido: int, fecha: date - estado: string + generarPedido(lista) + actualizarEstado(nuevo)

Descripción de la Aplicación y Ejecución

Es un programa sencillo para gestionar las ventas y el stock de la pinturería. Te ayuda a realizar las siguientes tareas diarias de forma rápida desde la pantalla:

- **Registrar nuevos productos:** Puedes ingresar artículos indicando su código (ID), nombre, marca, precio, cantidad disponible (stock) y color.
- **Consultar lista de precios:** Muestra el precio base de cada producto (ya con el 30% de ganancia de la pinturería) y te calcula automáticamente cuánto cobra la casa según cómo pague el cliente (efectivo aplica 10%descuento, tarjeta aplica un 15% de recargo)
- **Cobrar a los clientes:** Seleccionas los productos por su ID, indicas la cantidad que se llevan y el sistema se encarga de restar esa cantidad del stock. Si no hay suficiente stock, te avisa en pantalla.
- **Emitir el ticket de compra:** Al finalizar la venta, te genera en pantalla el desglose completo del cobro con el total definitivo a cobrar.

Una vez iniciado, se verá un menú con 4 opciones principales. Solo debes escribir el número de la opción que quieras usar y presionar Enter:

1. **Registrar producto:** Elige esta opción para ingresar un artículo nuevo llenando los datos que te vaya pidiendo el sistema.

2. **Mostrar productos y precios según método de pago:** Te muestra la lista completa de todos los productos cargados junto con sus precios finales para efectivo y tarjeta.

3. **Cobrar Productos:** Es la caja de cobro. Ingresas el ID del producto, la cantidad a vender, eliges el medio de pago (1 para Efectivo, 2 para Tarjeta) y el programa imprime el ticket final.

4. **Salir:** Cierra el sistema de forma segura.

Metodologías Aplicada y Evidencias

El equipo utilizó una metodología ágil XP (Extreme Programming), respetando las iteraciones fijadas y el seguimiento del trabajo a través de Trello. Trabajar con XP supuso adoptar un enfoque centrado en la calidad del código, la flexibilidad ante los cambios y la entrega constante de valor. En la práctica, esto implicó:

- **Desarrollo iterativo e incremental:** El proyecto se dividió en ciclos cortos de trabajo alineados con el calendario académico. En lugar de esperar al final para tener un producto completo, se construyeron versiones funcionales de forma progresiva, partiendo de un alcance mínimo operativo.

- **Priorización de la calidad y pruebas automatizadas**: Siguiendo la filosofía de XP de "probar antes o a la par de construir", se incorporaron pruebas con pytest para validar constantemente las reglas de negocio (como el cálculo de márgenes, descuentos y control de stock).

- **Simplicidad en el diseño:** Se aplicó el principio de *hacer lo más simple que pueda funcionar*, evitando sobrecargar el sistema y enfocándose estrictamente en resolver las necesidades reales de atención al público de la pinturería.

- **Transparencia y seguimiento visual:** Mediante el tablero de Trello se visibilizó el estado real de cada tarea e historia de usuario, facilitando la comunicación continua del equipo y la adaptación rápida ante cualquier imprevisto durante la iteración.

Como parte del seguimiento y la sincronización del equipo bajo la metodología ágil, se llevaron a cabo dos reuniones virtuales a través de Google Meet:

- **3 de septiembre de 2026:** Reunión de planificación para la definición de historias de usuario, asignación de tareas y organización del tablero en Trello.
- **24 de septiembre de 2026:** Reunión de revisión e integración para validar el avance del código, realizar pruebas funcionales con pytest y ajustar los detalles del entregable.

Gracias a estos pilares, el proyecto no solo logró entregas incrementales y funcionales, sino que mantuvo un código limpio, fácil de mantener y con un control constante sobre el avance.

Vinculo para el tablero de trello.
https://trello.com/b/hPjnbn8E/arcoiris

Vinculo para el historial de commits.
https://github.com/PastranaMatias/ArcoIris/commits/main/

Frameworks para Expansión Futura

Django

Con el propósito de escalar el sistema Arco-Iris de una aplicación de consola a una plataforma web robusta, multiusuario y lista para producción. La elección de Django responde a la necesidad de resolver limitaciones del alcance inicial (como el almacenamiento temporal en memoria) e incorporar capacidades avanzadas requeridas por el negocio:

- **Persistencia de Datos (ORM):** Las clases actuales del dominio (Producto, Ticket, Pedido y Usuario) se estructuran como **Modelos relacionales** de Django, permitiendo guardar la información de inventario y ventas de forma permanente en motores de base de datos como **SQLite** o **PostgreSQL**.

- **Gestión Administrativa (*Django Admin*):** Otorga de forma nativa un panel de control para que el personal de administración gestione el catálogo de productos, ajuste costos y configure parámetros de stock sin necesidad de desarrollar módulos desde cero.

- **Control de Acceso y Roles:** Integra el sistema de autenticación de Django para reemplazar el modelo Usuario actual, delimitando permisos específicos para los perfiles de **Atención al Público** (emisión de tickets) y **Depósito/Administración** (gestión de compras y precios).

- **Centralización de Reglas de Negocio:** La lógica contenida en la clase GestorProductos y las reglas de Descuento se trasladan a la capa de servicios y vistas del *framework*, garantizando un mantenimiento limpio del código.

- **Continuidad de Pruebas (XP):** Permite mantener la metodología de trabajo ágil y las pruebas automatizadas implementadas con pytest mediante la integración directa de la librería pytest-django.

Casos de uso de patrones de diseño

Para garantizar que el software sea escalable, mantenible y fácil de extender sin modificar el código existente, se contempla la incorporación de dos patrones de diseño de comportamiento:

Patrón Strategy permite definir una familia de algoritmos, encapsular cada uno en una clase independiente y hacerlos intercambiables en tiempo de ejecución.

- **Aplicación en el sistema:** Se utiliza para gestionar las distintas reglas de cálculo de precios y promociones. En lugar de usar múltiples condicionales (if/else) para determinar el costo según la forma de pago, se define una interfaz común (EstrategiaPrecio) con clases concretas como EfectivoStrategy (aplica un 10% de descuento) o CreditoStrategy (aplica un 15% de recargo).

- **Beneficio:** Permite agregar nuevos métodos de pago o promociones temporales (como un *Black Friday*) sin alterar la clase Producto ni la lógica de cobro.

Patrón Observer establece una relación de dependencia "uno a muchos", de modo que cuando un objeto cambia su estado, notifica automáticamente a todos sus suscriptores.

- Aplicación en el sistema: Se aplica sobre el control de inventario (Stock). Cuando se realiza una venta y el stock de un producto disminuye por debajo de un umbral mínimo, el objeto emite una notificación automática.

- Beneficio: Desacopla la venta de la gestión de inventario. Al recibir el aviso, un observador (AlertaObserver) muestra una advertencia en pantalla, mientras que otro (PedidoObserver) puede preparar automáticamente una solicitud de reposición al proveedor.

Refactorización del Código y Pruebas